OWL 2

Web 本体语言 2 · 规范速查

OWL 2 — OVERVIEW · MINDSET SWITCHES FIRST

PL. 04

W3C / OWL 2 / 2009·2012

01 总览 · RDF 之上的本体语言

PART 1 · OVERVIEW

OWL 2 在 RDF 上加本体表达:类构造子造类,属性特征加语义,公理说话,推理机按剖面算账。先开两大思维开关——开放世界、无唯一名。

语言定位	RDF 之上加本体表达 · 六份文档	开放世界
构件	类构造子十二式 · 属性特征七标签	造类的兵器
公理与推理	公理总清单 · 三剖面 · 复杂度	表达力换效率
思维与实践	两大开关 · 语法 · 工具 · 坑	先开开关再写

主线:开放世界假设 + 无唯一名假设——不切这两个开关,OWL 的每条结论都显得反直觉。

本卡结构 = 本图:概览 → 构件 → 公理 → 推理 → 实践。

02 家族地图 · 六份文档撑起一门语言

PART 1 · DOCUMENT FAMILY

OWL 2 于 **2009-10-27** 成为一套 W3C Recommendation,**2012-12-11** 发布第二版(技术内容不变)。它在 RDFS 之上补齐了逻辑武器:交集并集补集、量词、基数、属性特征、键、属性链。

文档	内容
Structural Spec / Functional Syntax	核心:结构、公理、函数式语法(本文主线)
Mapping to RDF Graphs	与 RDF 三元组互转的规则
Direct Semantics	DL 语义:对结构的直读模型论
RDF-Based Semantics	对 RDF 图的语义(OWL 2 Full,不可判定)
Conformance	一致性:DL/Full、语法限制
Profiles	三个子语言:EL / QL / RL(见 §14)
Primer / Quick Reference / New Features	入门、速查、与 1.1 的新旧对照

KEY POINT · 两种语义

Direct Semantics:把本体读成描述逻辑,可判定、推理机的主战场(OWL 2 DL)。**RDF-Based Semantics**:直接在 RDF 图上解释,表达力最大但**不可判定**(OWL 2 Full)。同一份文档,两种读法,先想清楚站哪边。

03 开放世界 + 无唯一名 · 两大思维开关

PART 1 · TWO MINDSET SWITCHES

开放世界假设 OWA

- 没写的≠没有——查不到 only C 的反例,不算违反
- 推不出「完整」:要闭合,显式写 `oneOf` / `maxCardinality 0`
- 「为什么推理机没推出」的第一答案:知识不全,或需显式闭包

无唯一名假设 no UNA

- 两个 IRI 默认**可能**是同一个体
- 要确认不同,写 `DifferentIndividuals`
- 要合并,写 `SameIndividual`;函数属性、键都会参与个体合并推理

典型翻车

「给了 maxCardinality 1 和两个不同的值,怎么不报错?」——因为推理机不知道那两个值不同!补 `DifferentIndividuals(a b)` 或改用函数属性+枚举,矛盾才浮出水面。

想要闭世界 · 四种显式闭合手段

手段	闭合什么
`ObjectOneOf(a b c)`	枚举:这个类的个体只有这几个
`maxCardinality 0`	显式禁止某属性取值
`DifferentIndividuals`	声明「这几个名字确指不同个体」
SHACL(PL.07) closed shapes	数据级校验的闭合,不动本体语义

OWL 2 Web 本体语言 2 · 规范速查

OWL 2 — LADDER & CLASS CONSTRUCTORS

PL. 04

W3C / OWL 2 / 2009·2012

04 语言全貌 · 一张图看表达力阶梯

PART 2 · EXPRESSIVITY LADDER

OWL 2 Full · RDF-Based Semantics · 不可判定

OWL 2 DL · Direct Semantics · N2EXPTIME

EL | **QL** | **RL**

生物本体 · PTIME　　数据库查询 · NLogSpace　　规则引擎 · PTIME

三剖面都是 DL 的语法子集,彼此互不包含

从下往上读:剖面用表达力换效率;OWL 2 DL 全量开火但推理贵;OWL 2 Full 只属于理论家。工程上先问一句话——**这个推理,多快的机器、多大的数据?**

OWL 2 相对 OWL 1 的新装备

装备	一句话
属性链	chain of properties 蕴含第三条——规则形状,仍可判定
HasKey	类级键,键值相同者推理为同一体
限定基数	基数可指定值的类(`min 2 hasParent Person`)
Self 限制	「p 指自己」入表达式
非对称 / 反自反	属性方向性特征补全
richer 数据类型	owl:rational/real、构面收窄值空间
注释体系	本体/实体级注解正式化,可自定义注解属性
三剖面	EL / QL / RL,表达力换效率的官方菜单

OWL 2 vs OWL 1

OWL 1 DL ≈ SHOIN(D),OWL 2 DL ≈ **SROIQ(D)**——多出的 R(链)、O(名)、I(逆)、Q(限定基数)正是上表的主干。

05 类构造子 · 造类的十二般兵器

PART 2 · CLASS CONSTRUCTORS

构造子	Functional Syntax	Turtle 缩写	读法
交集	`ObjectIntersectionOf(C D)`	`C ∩ D`(RDFS 语义可推)	既 C 且 D
并集	`ObjectUnionOf(C D)`	—	C 或 D
补集	`ObjectComplementOf(C)`	—	非 C
存在量词	`ObjectSomeValuesFrom(p C)`	`[p some C]*`	至少一个 p 是 C
全称量词	`ObjectAllValuesFrom(p C)`	`[p only C]*`	所有 p 都是 C(对没有 p 的个体也成立!)
具体值	`ObjectHasValue(p a)`	`[p value a]*`	p 恰为个体 a
自指	`ObjectHasSelf(p)`	—	p 指向自己
枚举	`ObjectOneOf(a b c)`	—	只能是这几个个体
基数(限定)	`ObjectMinCardinality(n p C)` / Max / Exact	`[p some/min/max n C]*`	p 到 C 的值至少/至多/恰好 n 个
数据版	`DataSomeValuesFrom(dp dr)` 等	—	同上,作用于数据属性
数据范围限制	`DatatypeRestriction(dt facet v)`	—	如 xsd:pattern、minInclusive
顶/底	`owl:Thing` / `owl:Nothing`	—	全集 / 空集

* Turtle 缩写

方括号写法是社区惯用的 Manchester-ish 简写,**不是** Turtle 语法本身——Turtle 里要写成 `owl:Restriction` 节点(见 §05 实例)。

06 全称量词的陷阱 · 空集也是「都」

PART 2 · UNIVERSAL QUANTIFIER TRAP

逻辑事实

「所有 p 都是 C」对**没有任何 p 的个体同样成立**——空集上的全称命题恒真。想让「必须至少有一个」,全称要与存在量词联用,或直接用基数 `min 1`。

Turtle 里限制的真实长相(标准 Restriction 节点)

```
:Pizza rdfs:subClassOf
  [ owl:onProperty :hasTopping ; owl:someValuesFrom :TomatoTopping ] ,  # 必须有番茄
  [ owl:onProperty :hasBase ; owl:allValuesFrom :ThinBase ] ,          # 只许薄底
  [ owl:onProperty :hasTopping ; owl:minQualifiedCardinality "3"^^xsd:nonNegativeInteger ;
    owl:onClass :Topping ] .                                           # 配料至少三种
```

限定 vs 非限定

`minCardinality` 只数个数,`minQualifiedCardinality` 还要求值的类型——限定位是 OWL 2 新增,建模时优先用它,信息量翻倍。

07 属性特征 · 七个标签一张表

PART 2 · PROPERTY CHARACTERISTICS

特征	Functional Syntax	语义	备注
传递	`TransitiveObjectProperty(p)`	p(x,y) ∧ p(y,z) → p(x,z)	祖先关系、部件包含
对称	`SymmetricObjectProperty(p)`	p(x,y) → p(y,x)	婚姻、邻居
非对称	`AsymmetricObjectProperty(p)`	p(x,y) → ¬p(y,x)	强于反自反;须简单属性
自反	`ReflexiveObjectProperty(p)`	全域个体 p(x,x)	注意是「全域自反」
反自反	`IrreflexiveObjectProperty(p)`	¬p(x,x)	父子、大于;须简单属性
函数	`FunctionalObjectProperty(p)`	p(x,y) ∧ p(x,z) → y=z	生日;数据属性同理
反函数	`InverseFunctionalObjectProperty(p)`	p(y,x) ∧ p(z,x) → y=z	社保号——值定主语,天然的「键」

简单属性(simple)限制

参与非对称、反自反、不相交、**基数限制**的属性必须是「简单属性」——不能处在属性链或传递闭包的圈里。这条全局限制正是可判定性的门票;违反即本体语法非法。

特征选型 · 常见关系该贴什么标签

关系	建议特征	理由
`:ancestorOf`	Transitive	传递关系原型
`:spouseOf`	Symmetric + Irreflexive	互为夫妻,不自己娶自己
`:parentOf`	Asymmetric + Irreflexive	方向不可逆
`:birthMother`	Functional	唯一性由事实保证
`:ssn`	InverseFunctional(数据属性亦可)	值定人,天然键
`:partOf`	视模型:传递 + 可选链	链 `partOf ∘ locatedIn ⊑ partOf` 常配对出现

08 属性链与键 · OWL 2 的两件新兵

PART 2 · PROPERTY CHAINS & KEYS

```
# 属性链:两条属性首尾相接,推出第三条(规则形状!可判定)
SubObjectPropertyOf(
  ObjectPropertyChain( :hasMother :hasSister )
  :hasAunt )
# 读法:有母亲 m,且 m 有姐妹 s,则 s 是姨母

# 键:同一键值(这里是身份证号)必属同一个人
HasKey( :Person :ssn )
# 语义:若两人 ssn 相同 → SameIndividual;注意键的推理只在「有名字的个体」间生效
```

链 vs SWRL

属性链是「如果…则…」最安全的一种:留在 DL 内、可判定、主流推理机全支持。需要变量比较、函数、否定判断的规则,才轮到 SWRL(PL.05)或规则引擎(剖面 RL)。

09 逆向属性 · 双向都写还是写一边

PART 2 · INVERSE PROPERTIES

```
InverseObjectProperties( :hasPart :partOf )
# 声明后写一边即可,另一边由推理补全——两处都写是冗余,漏写一边才有信息
```

Top / Bottom

`owl:topObjectProperty`(万物皆连)、`owl:bottomObjectProperty`(无人相连)、数据侧同款。bottom 与任何属性不相交,常用来「宣布不可能」。

10 公理总清单 · 类侧与个体侧

PART 3 · AXIOMS: CLASS SIDE

类公理

公理	Functional Syntax	读法
子类	`SubClassOf(C D)`	C 的实例都在 D 里
等价	`EquivalentClasses(C D E)`	互为子类(双向、可多条)
不相交	`DisjointClasses(C D E)`	两两无公共实例
不相交并	`DisjointUnion(C D E)`	C = D ⊔ E 且三者互斥(枚举类的标准写法)
注解公理	`AnnotationAssertion(rdfs:label C "…")`	挂元数据,Direct Semantics 不解释

```
# 枚举类的教科书写法:患者 = 男患者 ⊔ 女患者,且互斥
DisjointUnion( :Patient :ManPatient :WomanPatient )
# 等价于 EquivalentClasses + DisjointClasses 双公理,少写一半还防漏
```

个体断言(ABox)

断言	Functional Syntax	读法
类型	`ClassAssertion(C a)`	a 是 C 的实例
对象/数据属性	`ObjectPropertyAssertion(p a b)`	a p b
同一	`SameIndividual(a b)`	a、b 是同一个体(OWA 下不同名≠不同体!)
不同	`DifferentIndividuals(a b)`	a、b 确为两个个体
否定断言	`NegativeObjectPropertyAssertion(p a b)`	明确声明「a 不 p b」

提示

只有写进公理的东西推理机才知道。`DifferentIndividuals` 不写,两个 IRI 就**可能**是同一个体——开放世界里,名字不唯一。

11 公理总清单 · 属性侧

PART 3 · AXIOMS: PROPERTY SIDE

公理	Functional Syntax	读法
子属性/链	`SubObjectPropertyOf(p q)` / `(chain)`	p ⊑ q;链式见 §08
等价属性	`EquivalentObjectProperties(p q)`	互为子属性
逆属性	`InverseObjectProperties(p q)`	$p = q^{-1}$
定义域/值域	`ObjectPropertyDomain(p C)` / Range	与 RDFS 不同:**逻辑上强制**,违反可致不一致
不相交属性	`DisjointObjectProperties(p q r)`	同一对个体不能同时 p 又 q
特征声明	`Functional/Transitive/…Property(p)`	见 §07 全表

OWL 2 与 RDFS 的 domain/range 分野

RDFS 的 domain/range 只会**补类型**(引 PL.02);OWL 2 DL 里它们是逻辑公理——数据违反可推出矛盾(如个体既属 C 又属 ¬C),推理机报「不一致」。同一个词,两种强度,加载模式说了算。

```
# 同一声明,两种后果
ObjectPropertyDomain( :hasAuthor :Book )   # OWL 2 DL:hasAuthor 的主语必是 Book
:hasAuthor rdfs:domain :Book .              # RDFS 模式:主语被推成 Book,永不报错
# 数据::robot :hasAuthor :alice 且 :alice owl:differentFrom 任何 Book 实例
# → OWL DL 模式:本体不一致;RDFS 模式::robot a :Book,岁月静好
```

12 数据类型与面 · 值空间的边界

PART 3 · DATATYPES & FACETS

内容	清单
XSD 全家	xsd:string/integer/decimal/float/double/boolean/dateTime 等二十余种
RDF 侧	rdf:XMLLiteral(引 PL.01)
OWL 特供	`owl:real` / `owl:rational`;字面量 `"1"^^owl:rational`
构面 facets	length / minLength|maxLength / pattern / min|maxInclusive / min|maxExclusive / totalDigits / fractionDigits

```
# 「成年年龄」:整数且 ≥ 18 —— 用构面收紧值空间
SubClassOf( :Adult
  DataSomeValuesFrom( :age
    DatatypeRestriction( xsd:integer xsd:minInclusive "18"^^xsd:integer ) ) )
```

剖面的差别

RL 禁 owl:real / owl:rational;QL 只认一个精简清单。数据类型用错档位,本体就「越档」了——发布前用剖面校验器过一遍。

13 注解与 punning · 元数据的合法身份

PART 3 · ANNOTATIONS & PUNNING

注解 annotations

- `rdfs:label`、`rdfs:comment`、`owl:versionInfo`…挂在本体、类、属性、个体上
- Direct Semantics **不解释注解**——它们不影响推理,错了也不报不一致
- 要严格化元数据(如版本溯源),配 PROV-O(PL.11)

Punning —名多角

- OWL 2 DL 允许同一 IRI 在不同公理里当**类**又当**个体**(受限的多态)
- 例::Dog 既是一个类,又是「狗类」这个分类学概念个体
- 元建模的合法姿势,OWL 1 里要靠『0001』式歪招

14 三剖面 · 用表达力换效率

PART 4 · THREE PROFILES

剖面	为谁而生	复杂度(一致性等)	拿掉了什么
OWL 2 EL	超大规模术语表(SNOMED CT 级生物本体)	PTIME-complete	全称量词、并/补、基数、枚举(多个体)、属性特征大半;保留存在量词、链、传递
OWL 2 QL	大数据量、查询为主(DL-Lite;改写 SQL 打关系库)	NLogSpace-complete;查询改写 AC0	子类位的存在量词、Self、HasValue、枚举、全称、基数、属性链、函数/传递属性、键、SameIndividual
OWL 2 RL	规则引擎 / 三元组库上的可扩展推理(DLP 谱系)	PTIME-complete	只禁 DisjointUnion 与 ReflexiveObjectProperty,其余按位置设限(基数只许 0/1);附完整 RL/RDF 规则集

互相之间

三者都是 OWL 2 DL 的语法子集,但**彼此互不包含**(EL 有链,QL 有逆,RL 有负断言……)。选型先定任务:建模大术语表选 EL,库上查询选 QL,已有规则引擎选 RL。

每个剖面留了什么、丢了什么 · 抽样对照

构造/特征	EL	QL	RL
存在量词 someValuesFrom	**留(主力)**	仅 superclass 位	**留(受限)**
全称 allValuesFrom	丢	丢	**留(受限)**
并/补/枚举	丢	丢	限定位
属性链 / 传递	**留**	丢	丢(链)
逆属性	丢	**留**	**留(受限)**
基数	丢	丢	仅 0/1
负属性断言 / 不相交属性	丢	不相交留	**留**
键 HasKey	**留**	丢	丢

15 复杂度全景 · 推理的价目表

PART 4 · COMPLEXITY LANDSCAPE

语言	一致性/包含/实例检查(分类复杂度)	合取查询回答
OWL 2 DL(Direct Semantics)	N2EXPTIME-complete	可判定性尚开放(数据复杂度 NP-hard 起)
OWL 2 Full(RDF-Based Semantics)	**不可判定**	同上
OWL 2 EL	PTIME-complete	分类复杂度 EXPTIME
OWL 2 QL	NLogSpace-complete	NP-complete(合并)
OWL 2 RL	PTIME-complete	NP-complete(合并)
OWL 1 DL	NEXPTIME-complete	—

读表姿势

理论复杂度是「最坏情形」,好的优化器(增量大小的本体)日常飞快;但架构选型时,请把它当**信用上限**而不是性能承诺。

对应哪门 DL

OWL 2 DL 在描述逻辑谱系里约等于 **SROIQ(D)**(S=传递属性,H=子类,O=枚举,I=逆,N=基数,ROIQ 为 OWL 2 扩展,D=数据类型);OWL 1 DL 是 SHOIN(D)。看论文里的黑体缩写,对号入座。

16 推理任务五件套 · 推理机到底在算什么

PART 4 · FIVE REASONING TASKS

任务	问的问题	失败形态
一致性	整本本体有没有矛盾?	inconsistent ontology
可满足性	这个类可能有实例吗?	unsatisfiable class(常见于 domain 滥用)
包含	C ⊑ D 吗?	层级树重排
实例检查	a 是 C 吗?	类型补全
实现	a 在最具体的哪个类?	个体精确归位

一致 != 正确

推理机只担保「逻辑上没矛盾」,不担保「业务上没错」——垃圾公理可以完美自洽。校验数据结构请配 SHACL(PL.07),两者是互补不是替代。

17 不一致急救流程 · 推理机报错之后

PART 4 · INCONSISTENCY FIRST AID

01 先定位本体还是数据:只加载 TBox 跑一致性——TBox 干净就查 ABox 断言。

02 二分公理:注释掉一半再跑,迭代逼近肇事公理(Protégé 的 explain 功能可自动做)。

03 常见肇事者:domain/range 滥用、缺 DifferentIndividuals 的基数冲突、把校验需求错写进 OWL。

04 修完跑剖面校验:确定本体仍在目标剖面(EL/QL/RL)内,再上线。

18 同一本体的两种写法 · Turtle vs Functional

PART 5 · TURTLE VS FUNCTIONAL

```
# ============ Turtle(工具生态常用) ============
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix :     <http://example.org/onto#> .

:Dog a owl:Class ;
    rdfs:subClassOf
      [ a owl:Restriction ;
        owl:onProperty :hasPart ;
        owl:someValuesFrom :Bone ] ,          # 有骨头
      [ a owl:Restriction ;
        owl:onProperty :hasParent ;
        owl:maxQualifiedCardinality "1"^^xsd:nonNegativeInteger ;
        owl:onClass :Dog ] .                   # 至多一个狗爸爸

:hasPart a owl:TransitiveProperty .
:Dog owl:disjointWith :Cat .
:rex a :Dog ; :hasPart :rexTail .
```

```
# ==== Functional-Style Syntax(规范原文/论文常用) ====
SubClassOf( :Dog ObjectSomeValuesFrom( :hasPart :Bone ) )
SubClassOf( :Dog ObjectMaxCardinality( 1 :hasParent :Dog ) )
TransitiveObjectProperty( :hasPart )
DisjointClasses( :Dog :Cat )
ClassAssertion( :Dog :rex )
ObjectPropertyAssertion( :hasPart :rex :rexTail )
SubObjectPropertyOf( ObjectPropertyChain( :hasMother :hasSister ) :hasAunt )
HasKey( :Person :ssn )
SubClassOf( :Cat ObjectUnionOf( :HouseCat :WildCat ) )
DifferentIndividuals( :rex :whiskers )
NegativeObjectPropertyAssertion( :eats :rex :whiskers )
```

读法

两行一对照:Restriction 节点 = 构造子;`rdfs:subClassOf` = SubClassOf。规范与论文用 Functional,数据交换用 Turtle/RDF 映射——两边无损互转。

19 本体的自我管理 · 版本、导入与弃用

PART 5 · ONTOLOGY MANAGEMENT

机制	词汇/写法	用途
本体头	`owl:Ontology` + `owl:versionIRI`	本体的 IRIs;version IRI 标识特定版本
版本关系	`owl:backwardCompatibleWith` / `incompatibleWith` / `priorVersion`	版本链;兼容性承诺
导入	`owl:imports`	引另一本体的全部公理;Direct Semantics 按导入闭包解释
弃用	`owl:deprecated`	标记弃用实体(布尔注解)
自描述注解	`owl:AnnotationProperty` / `owl:versionInfo` / `owl:priorVersion`	注解体系本身可扩展

`imports 不是 include`
导入是**公理并集**:对方改版,你的推理结果跟着变。生产环境钉住 version IRI 导入,别用 latest——PL.02 讲过的「原地覆盖」是分布式推理的定时炸弹。

```
# 本体头的标准写法
<http://example.org/onto/1.0> a owl:Ontology ;
    owl:versionIRI <http://example.org/onto/1.0> ;
    owl:imports <http://www.w3.org/2004/02/skos/core> ;
    owl:priorVersion <http://example.org/onto/0.9> ;
    rdfs:label "领域本体 v1.0"@zh-CN .
```

20 工具速览 · 推理机从哪进

PART 5 · TOOLING

工具	路线
Protégé	事实标准编辑器;explain、剖面校验、本体可视化
HermiT / Openllet(Pellet)	Tableau 全量 DL 推理机,支持 SWRL
ELK	EL 剖面专用,超大术语表秒级分类
Jena rules / GraphDB / RDFox	RL 谱系:规则引擎/三元组库上的可扩展推理
OWL API (Java)	读写 Functional/Turtle/RDF 三形态的库底座

21 实操坑清单

PART 5 · PITFALLS

01 全称量词对「没有 p 的个体」恒真——要「必须有」,用 min 1 或与 someValuesFrom 联用。

02 没写 DifferentIndividuals,两个 IRI 可能是同一个人——基数冲突不报错,多半是缺不同声明。

03 domain/range 在 OWL 2 DL 里是硬公理,违反即不一致——RDFS 的「补类型」直觉在这里会害你。

04 非对称/反自反/不相交/基数参与者必须是**简单属性**——挂过链或传递的属性越界即语法非法。

05 hasKey 只在有名字的个体间触发推理,别当数据库唯一约束用;真校验去 SHACL。

06 OWA 下「闭世界式」需求要显式闭合:oneOf、max 0、differentFrom。

07 选剖面先选任务:大术语表 EL、库上查询 QL、规则引擎 RL;剖面互不包含,别幻想通吃。

08 owl:imports 是公理并集,对方改版你遭殃——钉 version IRI。

09 注解不参与 Direct Semantics:标签写错不报错,发布前人工过。

10 复杂度是上限不是承诺:DL 推理在病态本体上指数爆炸,大 ABox 优先考虑 QL/RL 路线。

11 punning 有度:同一 IRI 的多角色要服务同一叙事,否则读本体的人比推理机先崩溃。

12 空表达式与 owl:Thing 滥用:把 owl:Thing 写进限制位、或造出必然为空的类,推理机轻则告警重则 unsatisfiable。

13 构面 facet 只能收紧已知类型:pattern 配 string、minInclusive 配数值——张冠李戴即越出数据类型词表。

14 DL 模式与 Full 模式混读:文档同时被两种语义解释,团队讨论先统一「今天在哪个语义里说话」。

15 发布前跑剖面校验器:表达力悄悄越档(多用了一个基数),下游 QL/RL 工具直接拒载。